第一部分：核心併發與庫存邏輯 (Concurrency & Inventory)

1. 併發控制機制：如何確保只有一個人成功？

- **技術實作：** 系統使用了 `DbContextTransaction`。當使用者發起鎖位請求時，後端會開啟一個資料庫事務，並在 SQL Server 內對該座位資料進行狀態變更（從 `Available` 改為 `Locked`）。
- **面試回答：** 「我利用了資料庫的事務原子性。在 `Seat` 狀態變更的瞬間，SQL Server 會對該行資料施加排他鎖，確保在同一毫秒內，即使有萬人點擊，也只有第一個事務能成功執行 Update 並 Commit。」

2. 鎖定機制選型：為何選資料庫鎖？

- **技術實作：** 程式碼中直接操作 `TicketsContext` 進行資料更新。
- **面試回答：** 「選擇資料庫鎖（Database-level locking）是因為它是系統的**單一信任來源**。雖然應用程式層級的 lock 在單機開發時有效，但考慮到未來系統若部署在負載平衡（Load Balancer）後的多台伺服器，資料庫鎖能確保跨節點的一致性，防止超賣。」

3. 隔離層級 (Isolation Level) 與原子性

- **技術實作：** 採用 ASP.NET Core 預設的事務處理。
- **面試回答：** 「在搶票邏輯中，我確保『查詢餘票』與『扣除庫存』是在同一個 Transaction 範圍內完成。這防止了 A 使用者看到有票，但在按下購票前 B 使用者已經把票賣走的情況（即避免『幻讀』或『不可重複讀』對業務邏輯的影響）。」

4. 佔位鎖定流程與超時處理 (核心亮點)

- **技術實作：** `Orders` 表中有 `LockTime`，並搭配 `TicketCleanupService`。
- **面試回答：** 「當使用者點選座位，系統會進入『鎖定狀態』並記錄時間戳記。我實作了一個 **Background Service**，它會每分鐘輪詢資料庫，自動將鎖定超過 10 分鐘且未付款的訂單撤銷，並將座位重新釋回市場。這確保了庫存不會因為使用者關閉網頁而被永久佔用。」

第二部分：資料庫設計與效能優化 (DB & Performance)

1. 預存程序 (Stored Procedures) 的應用

- **技術實作：** 雖然主體使用 EF Core，但在文件中有提到 Stored Procedure。
- **面試回答：** 「對於高頻率、邏輯較複雜的批次操作，我選擇使用預存程序。這能減少 Web 伺服器與資料庫伺服器之間的來回通訊（Round-trip），且 SQL Server 能針對這些預編譯的指令進行執行計畫優化。」

2. 索引策略與查詢優化

- **技術實作：** 針對活動搜尋（Keyword）與場次篩選。
- **面試回答：** 「我在 `Activities` 的關鍵字欄位與 `Shows` 的日期欄位建立了索引。這在面對數千場演唱會查詢時，能大幅降低全表掃描（Full Table Scan）帶來的效能延遲。」

第三部分：後端架構與背景服務 (Architecture)

1. 背景服務 (Worker Service) 的穩定性

- **技術實作：** `TicketSalesSystem` 中的 `Worker` 類。
- **面試回答：** 「我的背景服務是繼承自 `.NET BackgroundService`。它與主程式生命週期綁定，且透過注入 `IServiceScopeFactory` 來存取資料庫。即便伺服器重啟，由於所有過期邏輯是基於資料庫時間戳記，服務啟動後會立即恢復並處理未完成的回收任務。」

2. DTO 與 ViewModel 的區隔

- **技術實作：** 程式碼中定義了 `DTOs` 與 `ViewModels` 資料夾。
- **面試回答：** 「我嚴格區分了資料傳輸對象（DTO）與視圖模型（ViewModel）。DTO 用於 API 通訊，隱藏了資料庫實體的敏感資訊（如 UserID 或狀態碼）；ViewModel 則專注於前端顯示邏輯，這提升了系統的安全性和程式碼的可讀性。」

第四部分：行動端整合 (Mobile & QR Code)

1. QR Code 驗票與安全

- **技術實作：** Android `ScannerActivity` 結合 ML Kit。
- **面試回答：** 「行動端是我系統的閉環關鍵。透過 Google ML Kit 進行本機影像處理，能確保在掃描瞬間極速反應。驗票時，App 會傳送票券雜湊值到後端，資料庫會驗證 `IsUsed` 狀態，防止截圖或二次進場的問題。」

2. 行動端架構設計

- **技術實作：** 採用 Proxy 模式介接 API。
- **面試回答：** 「為了確保 Android 端能快速同步後端變更，我採用了代理（Proxy）架構，讓行動裝置專注於 UI 呈現與硬體掃描，複雜的業務驗證邏輯則統一由 ASP.NET Core 後端處理。」

第五部分：未來優化 (Scalability)

注意： 這是針對你提到的「程式碼沒 Redis，但文件有寫」的回應方式。

1. 關於 Redis 的定位

- **面試回答：** 「在目前的實作中，我優先確保了**資料的絕對一致性**（使用 SQL Transaction）。但我已經在架構設計中預留了 Redis 的擴展空間。當系統面臨每秒萬次以上的『熱門活動查詢』時，我會將靜態的活動資訊緩存在 Redis，並利用 Redis 的 `Atomic Decrement` 來處理初期的流量削峰，最後才落庫到 SQL Server。」

2. 黃牛防治與防禦

- **面試回答：** 「目前透過背景服務清理惡意佔位，未來預計在 `LockSeat` API 加入 **Rate Limiting (流量限制)** 與 **圖形驗證碼**。這能從技術層面增加黃牛腳本的開發成本，保護一般使用者的購票權利。」

一、 專案架構與開發動機 (Project Overview)

這部分通常是面試的開場白，測試你的溝通能力與對專案的熟悉度。

1. **專案核心價值：** 請用一分鐘介紹這個售票系統解決了什麼痛點？

   確保購票公平性（解決超賣與佔位）：

   針對瞬間高併發流量，我透過資料庫事務（Transaction）確保庫存扣除的原子性，徹底杜絕『付了錢卻沒票』的超賣現象。同時，我實作了自動化庫存回收服務（Background Service），每分鐘清理逾時未付款的訂單。這解決了惡意佔位導致真誠買家買不到票的痛點，讓票券資源能高效循環。

   打造營運閉環（解決驗票效率）：

   不同於多數只做網頁的專案，我開發了 Android 驗票端。利用 Google ML Kit 的高速辨識能力，讓手機直接變成場館驗票機。這解決了傳統紙本驗票效率低、易偽造的問題，實現了從『線上購票、數位儲存』到『現場秒級驗票』的完整 O2O 閉環。

2. **技術選型原因：** 為什麼選擇 ASP.NET Core 搭配 SQL Server，而不是 Node.js 或其他資料庫？
3. **開發時程控管：** 這半年（2025/10 - 2026/03）的開發節奏是如何安排的？最耗時的是哪個階段？
4. **系統角色劃分：** 管理者、消費者與行動端驗票員在系統中的互動邏輯為何？
5. **Clean Architecture：** 你的專案目錄（Controllers, DTOs, Services, Models）是依據什麼原則分類的？

二、 核心併發與交易處理 (Concurrency & Transactions)

這是售票系統的命脈，面試官一定會深挖。

6. **防止超賣機制：** 在程式碼中，你是如何確保「最後一張票」不會被兩個人同時買到？
7. **Transaction 範圍：** 你的 `DbContextTransaction` 是從哪裡開始，到哪裡 Commit？中間出錯怎麼辦？
8. **隔離層級選取：** SQL Server 預設是 Read Committed，對於搶票場景這夠嗎？你有考慮過 Serializable 嗎？
9. **Lock 的效能代價：** 使用資料庫鎖（Database Lock）時，如果請求量太大導致鎖定時間（Lock Time）過長，你會如何優化？
10. **樂觀鎖 vs 悲觀鎖：** 你的系統目前偏向哪一種？在「查詢餘票」時，你會建議用哪一種？

三、 資料庫設計與正規化 (Database Design)

測試你對資料結構與關聯的理解。

11. **ERD 設計邏輯：** 為什麼要設計超過 15 張表？請舉例說明哪兩張表的關聯最關鍵？
12. **Activity vs Show：** 為什麼「活動」與「場次」要拆分？一對多的設計考量為何？
13. **資料一致性：** 票券狀態（Seat Status）與訂單狀態（Order Status）如何保持同步？
14. **反正規化考量：** 為了提升查詢活動列表的速度，你有在任何地方進行反正規化（冗餘儲存）嗎？
15. **預存程序 (Stored Procedure)：** 哪些邏輯你寫在 C#，哪些寫在 SQL SP？判斷標準是什麼？

四、 背景服務與自動化 (Background Services)

這是你專案中的技術亮點之一。

16. **TicketCleanupService：** 這個背景服務的觸發頻率是多少？如果設定太快或太慢會有什麼影響？
17. **非同步輪詢：** 背景服務執行時，如果資料庫正在進行大流量寫入，會造成衝突嗎？
18. **重啟復原：** 如果伺服器當機重啟，背景服務如何知道哪些「逾時訂單」還沒被處理？
19. **Worker 的相依性注入：** 在 Singleton 的背景服務中，你是如何獲取 Scoped 的 `DbContext`？
20. **未來優化：** 除了清理訂單，背景服務還能用來處理什麼任務？（如：發送購票成功通知）。

五、 ASP.NET Core 核心技術 (C# & Framework)

測試你的基本功與對框架的掌握。

21. **Async/Await：** 為什麼 API 建議使用非同步？這對 Web Server 的 Thread Pool 有什麼幫助？
22. **Dependency Injection：** 請解釋 `AddScoped` 與 `AddTransient` 在你專案中的具體應用場景。
23. **Entity Framework Core (EF Core)：** 你是用 Code First 還是 Database First？當資料結構改變時，你如何進行 Migration？
24. **DTO 的必要性：** 為什麼不直接把資料庫的 Model 傳給前端？舉一個 DTO 保護資訊安全的例子。
25. **Middleware：** 你有實作任何中間件嗎？例如全域的 Exception Handling。

六、 Android 與行動端整合 (Mobile & ML Kit)

這是你與一般後端工程師拉開差距的部分。

26. **ML Kit 整合：** 為什麼選擇 Google ML Kit？它與傳統的 ZXing 掃碼庫相比有什麼優勢？
27. **API 通訊：** Android 端是如何處理 API 請求失敗（例如沒網路）的情況？你有實作 Retry 機制嗎？
28. **Proxy 模式：** 文件中提到 Android 端採用 Proxy 架構，這解決了什麼具體問題？
29. **驗票安全性：** 掃碼後，App 如何確保這張票不是「截圖」或「偽造的 QR Code」？
30. **硬體調度：** 在 Android 中如何優化相機預覽（Camera Preview）的效能與省電？

七、 系統安全與防護 (Security)

31. **身分驗證 (Authentication)：** 你的 API 是如何驗證使用者身分的？（Cookie vs JWT）。
32. **SQL Injection：** EF Core 如何幫你防止 SQL 注入？在寫原生 SQL 或 SP 時你如何預防？
33. **XSS 防範：** 當管理者在後台輸入活動描述時，你如何防止惡意腳本注入？

34. **敏感資料處理：** 使用者的密碼在資料庫中是明碼嗎？你用了哪種雜湊演算法？
35. **越權存取：** 如何確保 A 使用者無法透過修改 URL 看到 B 使用者的訂單內容？

八、 效能優化與擴展性 (Scalability)

36. **快取策略：** 活動清單（Home Data）如果每秒萬人讀取，你會如何引入 Cache 機制？
37. **N+1 問題：** 在讀取活動及其關聯的場次時，你如何避免產生過多的 SQL 查詢？（Include 的應用）。
38. **流量削峰 (Load Leveling)：** 如果開賣瞬間流量爆發，除了加伺服器，你覺得引入訊息佇列（MQ）有幫助嗎？
39. **Redis 的位置：** 如果要加入 Redis，你會用它來存什麼？（座位狀態、Session 還是活動資訊？）。
40. **索引優化：** 你如何找出哪些 SQL 查詢太慢？你會看執行計畫（Execution Plan）嗎？

九、 錯誤處理與測試 (Debugging & Testing)

41. **日誌管理 (Logging)：** 系統出錯時，你如何得知是哪個環節出了問題？
42. **單元測試：** 如果要幫「鎖定座位」寫測試案例，你會測試哪些邊緣情況（Edge Cases）？
43. **模擬搶票：** 你如何測試系統在高併發下的表現？有使用過工具（如 JMeter 或 Postman Runner）嗎？
44. **InnerException：** 在處理 try-catch 時，為什麼要注意 InnerException？

十、 個人成長與軟實力 (Personal Growth)

45. **最困難的挑戰：** 整個專案中，哪個功能讓你卡住最久？你最後是怎麼解決的？
46. **技術債：** 如果現在多給你一個月，你最想重構（Refactor）哪一段程式碼？
47. **學習方法：** 在開發過程中遇到不懂的技術（如 ML Kit），你是如何快速學會並應用的？

- **優化手段：** 針對高頻率查詢（如活動清單、關鍵字搜尋），我在 ActivityName 與 ShowDate 欄位建立了**非叢集索引（Non-clustered Index）**，將全表掃描轉為索引搜尋。

Q5：為什麼已經用了 EF Core，還要寫預存程序 (Stored Procedure)？

- **標準答案：** EF Core 雖然開發快速，但在處理極高頻率的批次更新（如一次處理上百張票券回收）時，網路傳輸與轉換開銷較大。預存程序在資料庫端編譯，執行計畫穩定，能更精準地控制鎖定範圍，減少 Lock Time。

三、 背景服務 (Worker Service)

Q6：背景服務 TicketCleanupService 的實作邏輯與價值為何？

- **標準答案：** 這是為了解決「佔位不買」的資源浪費。
- **實作細節：** 繼承自 .NET BackgroundService，每分鐘啟動一次。它會掃描 Orders 表中狀態為「未付款」且 CreateTime 超過 10 分鐘的紀錄，將關聯的 Seats 狀態還原。這確保了有限的票券資源能不斷循環利用。

Q7：如果背景服務執行時伺服器重啟了怎麼辦？

- **標準答案：** 邏輯是基於「資料庫內的紀錄」而非記憶體。只要服務重新啟動，它會根據 SQL 內的時間戳記繼續處理未完成的釋放任務，具備極高的容錯性（Resilience）。

四、 行動端整合 (Android & ML Kit)

Q8：為什麼選擇 Google ML Kit 實作掃碼？

- **標準答案：** ML Kit 是基於 Google 的機器學習模型，辨識速度快、容錯率高。相比傳統的 ZXing，它在光線不足、反光或 QR Code 稍微破損的情況下，依然能精準讀取，這對演唱會場館的快速驗票至關重要。

Q9：Android 端採用 Proxy/Wrapper 架構解決了什麼問題？

- **標準答案：** 在短時程開發下，Android 與後端 API 的資料結構往往需要頻繁微調。透過代理模式（Proxy），我讓 Android 端專注於 UI 與硬體溝通，將複雜的網路請求與例外處理封裝起來，大幅降低了雙端的耦合度與開發時災難。

---

48. **職涯銜接：** 你過去在領班與行政職位的經驗，如何轉化為開發售票系統時的邏輯優勢？（例如：對流程的嚴謹要求）。

💡 **準備建議：**

面試官不一定會問「標準答案」，他們更喜歡問**「如果是你，你會怎麼做？」**。

- 針對 **「併發控制」** 與 **「行動端驗票」** 這兩個部分，建議你準備好對應的**程式碼片段**，隨時準備開螢幕解釋。
- 針對 **「Redis」** 與 **「效能優化」**，即使程式碼沒寫，也要能說出一套「升級路徑」，這展現了你的潛力。

這份題庫幾乎涵蓋了 90% 以上的售票系統面試考點。**你想要我針對哪幾個特定的問題，幫你撰寫「範本級」的回答嗎？**

五、 軟體工程品質 (Clean Code & Security)

Q10：為什麼要使用 DTO (Data Transfer Object)？

- **標準答案：** 1. **安全性：** 避免直接回傳資料庫 Model（可能包含密碼雜湊或內部狀態）。
  2. **效能：** 只傳送前端需要的欄位，減少 JSON 封包大小。
  3. **解耦：** 當資料庫 Schema 改變時，前端 API 介面可以維持穩定。

Q11：如何防範常見的 Web 攻擊（如 SQL 注入、越權存取）？

- **標準答案：** * **SQL 注入：** 全面使用 EF Core 參數化查詢，不使用字串拼接 SQL。
  - **越權存取（IDOR）：** 在「查看訂單」API 中，後端會從 Claims（憑證）取回當前登入使用者的 ID，並比對訂單擁有者，確保 A 使用者無法透過修改 URL 看到 B 使用者的資料。

六、 未來優化 (未來展望 - 即使沒寫也要懂)

Q12：如果現在流量增加 100 倍，你的系統瓶頸會在哪？如何解決？

- **標準答案：** 瓶頸會在 **資料庫的 I/O 與鎖競爭**。
- **優化方案：** 1. 引入 **Redis 快取**：將「剩餘票數」存在 Redis，利用記憶體的高速讀寫先行扣除，非同步再寫回資料庫。
  2. 引入 **Message Queue (RabbitMQ)**：將訂單寫入非同步化（流量削峰），避免瞬間流量衝垮資料庫。

七、 個人成長題 (Soft Skills)

Q13：開發這個專案最困難的 Bug 是什麼？如何解決？

- **標準答案：** 曾遇到「非同步請求導致的資料庫連線逾時」。透過追蹤發現是事務鎖定範圍過大。我後來將非關鍵的查詢移出 Transaction，並優化了索引，成功將 API 回傳時間縮短了 50%。

Q14：你過去的行政/領班經驗，對你轉職工程師有什麼幫助？

- **標準答案：** 行政工作訓練了我對**流程的嚴謹度**，領班經驗則讓我懂得**團隊協作與溝通**。在寫售票系統時，我能從運作流程（如管理者的排班、消費者的防呆）出發去設計代碼，而不只是盲目寫 code，這讓我的程式邏輯更貼近真實業務場景。

這是**一份針對你售票系統專案的全方位面試標準答案集**。我將答案與你提供的 Project_Full_Code.txt 實作邏輯以及 票務銷售系統.docx 的設計理念完全對接。

這份資料分為：**核心併發、資料庫設計、背景服務、前端與行動端、安全性與優化**。

一、 核心併發與搶票邏輯 (Concurrency Control)

Q1：在高併發（秒殺）場景下，如何確保票券不超賣、不重賣？

- **標準答案：** 我在資料庫層級使用了 **DbContextTransaction** 結合 SQL Server 的事務原子性。當使用者發起鎖位請求時，系統會開啟一個 Transaction，將該座位的 Status 從「可售」改為「鎖定中」。
- **技術細節：** 即使萬人同時點擊，SQL Server 的排他鎖（Exclusive Lock）也會強制請求排隊，確保只有第一個成功變更狀態的事務能提交（Commit），其餘事務會因狀態已被變更而失敗。

Q2：為什麼選擇資料庫鎖（Database Lock）而不是程式碼鎖（C# lock）？

- **標準答案：** 因為 lock 語法只能限制單一 Web 伺服器內的執行緒。在真實環境中，系統通常會部署在多台伺服器（Load Balance）後方，此時只有**資料庫作為單一信任來源**才能確保跨伺服器的資料一致性，徹底防止超賣。

Q3：請說明「佔位鎖定」到「訂單成立」的完整流程？

- **標準答案：** 1. 使用者選擇座位，呼叫 API 開啟事務。
  2. 更新 Seats 表狀態並寫入 Order 表，記錄 LockTime（鎖定時間戳記）。
  3. 事務提交，座位進入「預訂」狀態，使用者進入付款頁。
  4. 若 10 分鐘內未付款，背景服務會自動釋放該座位。

二、 資料庫設計與效能 (Database & SQL)

Q4：你的資料庫設計超過 15 張表，如何平衡正規化與效能？

- **標準答案：** 核心邏輯遵循 **第三正規化 (3NF)** 以確保資料不冗餘。例如將「活動（Activity）」與「場次（Show）」分開，是因為同一個活動可能有不同日期與場館。

💡 **給你的最後提醒：**

1. **畫圖回答：** 面試官問到併發時，直接在紙上畫出「User -> API -> SQL Transaction -> Commit」的流程，專業度直接滿分。
2. **程式碼隨時備查：** 如果面試官問 TicketCleanupService 怎麼寫的，你可以大方說：「我這部分的邏輯是每分鐘掃描一次資料庫，我有帶程式碼可以解釋具體實作。」

這套答案集覆蓋了你提供的所有資料。祝你面試順利！